AI-Driven Proctored Exam Platform

A Major Project report for the evaluation and partial fulfillment of the requirement for the award of the degree

B. TECH. (CSE)

Submitted by –

SHLOK GUPTA (225/UCF/103)

MILAN KUMAR (225/UCF/071)

LUCKY GARG (225/UCF/068)

LAXMI NARAYAN SHARMA (225/UCF/067)

UNDER THE SUPERVISION OF

Dr. RAKESH KUMAR

UNIVERSITY SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

GAUTAM BUDDHA UNIVERSITY

GREATER NOIDA – 201312, GAUTAM BUDDHA NAGAR, UTTAR PRADESH, INDIA

FEB. 2026

Candidate's Declaration

We, hereby, certify that the work embodied in this project report entitled "**AI-Driven Proctored Exam Platform**" in partial fulfillment of the requirements for the award of the Degree of B. Tech. (CSE) submitted to the School of Information and Communication Technology, Gautam Buddha University, Greater Noida is an authentic record of our own work carried out under the supervision of Dr. Rakesh Kumar, School of ICT. The matter presented in this report has not been submitted in any other University / Institute for the award of any other degree or diploma. Responsibility for any plagiarism related issue stands solely with us.

Name	Roll No	Signature
SHLOK GUPTA	225/UCF/103	
MILAN KUMAR	225/UCF/071	
LUCKY GARG	225/UCF/068	
LAXMI NARAYAN SHARMA	225/UCF/067	

This is to certify that the above statement made by the candidates is correct to the best of my knowledge and belief. However, responsibility for any plagiarism related issue solely stands with the students.

Signature of the Supervisor: _____

Dr. Rakesh Kumar, School of ICT, Gautam Buddha University

Acknowledgement

We have taken considerable efforts in completing this project. However, it would not have been possible without the kind support and help of many individuals and organisations. We would like to extend our sincere thanks to all of them.

We are highly indebted to **Dr. Rakesh Kumar** for his guidance, constant supervision, and for providing the necessary direction throughout the development of this project.

We would like to thank **Dean of ICT Dr. Arpit Bhardwaj** and **HOD of CSE, Dr. Arun Solanki**, whose encouragement and institutional support were invaluable. We also express gratitude to our friends and family members whose cooperation helped us in the completion of this work.

Abstract

This project presents the design, architecture, and implementation of an **AI-Driven Proctored Exam Platform** developed for Gautam Buddha University (GBU), Greater Noida. The platform is a full-stack web application that enables faculty and administrators to create, schedule, and manage online examinations while continuously monitoring student behaviour through real-time artificial intelligence.

The backend is implemented using **Spring Boot 3.3.4 (Java 17)** with JWT-based authentication, while the frontend is built as a React 18 single-page application using Vite and Tailwind CSS. A dedicated **Python FastAPI** microservice hosts the AI proctoring pipeline, which employs **YOLOv8n** for object detection, **MediaPipe FaceMesh** with OpenCV solvePnP for gaze and head-pose estimation, **webrtcvad** for voice activity detection, the **face_recognition** library (Dlib) for identity verification, and an **XGBoost** classifier for behavioural risk scoring. All services are containerised using **Docker Compose** and communicate via **RabbitMQ** message queues, with **PostgreSQL 16** as the primary database, **Redis** for token management, and **MinIO** for object storage.

Key proctoring capabilities include face presence detection, multi-face alerting, gaze tracking, head pose estimation, lip-movement analysis, phone and notes detection, voice activity detection, and browser-behaviour monitoring (tab switches, fullscreen exits, copy-paste attempts). Violations are weighted and aggregated into a composite risk score; proctors receive real-time alerts via WebSocket. The platform has been functionally tested end-to-end across multiple simulated exam sessions.

Table of Contents

Chapter 1 Introduction	1
Chapter 2 Literature Review	4
Chapter 3 System Analysis	6
Chapter 4 System Design & Architecture	9
Chapter 5 Cheating Detection Modules	14
Chapter 6 Technology Stack & Implementation	18
Chapter 7 Security, Privacy & Ethics	21
Chapter 8 Evaluation & Testing	23
Chapter 9 Future Scope	24
Chapter 10 Conclusion	25
References	26

Chapter 1: Introduction

1.1 Background and Motivation

The rapid expansion of online education and remote assessment has created an urgent need for robust examination proctoring solutions. Universities and certification bodies worldwide now conduct a significant portion of their assessments digitally, requiring reliable systems to maintain academic integrity without the physical presence of invigilators.

Traditional supervision methods are fundamentally inadequate for remote settings. Students can exploit the lack of oversight by using mobile phones to search for answers, switching browser tabs to access unauthorised resources, using notes or textbooks, or having another person assist them. Existing commercial proctoring solutions such as ProctorU and ExamSoft are prohibitively expensive for many institutions, particularly in developing countries, and often raise serious privacy concerns.

This project addresses these challenges by presenting a fully implemented, open-source **AI-Driven Proctored Exam Platform** built for **Gautam Buddha University (GBU)**, Greater Noida. The platform combines a robust exam management system with a real-time AI proctoring pipeline, enabling faculty to conduct secure online examinations without relying on expensive third-party services.

1.2 Problem Statement

The existing landscape of online examination proctoring suffers from several critical deficiencies. Most simple systems only record video for post-exam review without performing real-time analysis, meaning cheating goes undetected during the exam. Commercial solutions carry significant licensing fees and vendor lock-in. Few systems analyse multiple behavioural channels simultaneously — video, audio, browser activity, and input patterns — and there is no comprehensive open-source alternative that combines all necessary detection modules into a production-ready, containerised platform.

1.3 Proposed Solution

This project proposes and delivers a complete AI-driven proctoring platform comprising a Spring Boot REST API backend, a React SPA frontend, and a Python FastAPI AI service. The system supports three user roles — **Admin**, **Proctor**, and **Student** — each with dedicated dashboards and workflows. Exams are conducted in a locked, full-screen browser environment; the AI service continuously analyses the webcam feed, microphone audio, and keyboard/mouse behaviour, generating proctoring events that proctors can review live or post-exam.

1.4 Project Objectives

Primary Objectives

- Provide a secure online examination system for university use with role-based access control.
- Automate invigilation using AI-based face detection, gaze tracking, object detection, lip-movement monitoring, and audio activity detection.

• Detect and log violations in real time (phone, notes, multiple faces, gaze-away, suspicious audio, tab switches, fullscreen exits) and alert proctors via a live monitoring dashboard.

• Support identity verification by matching a student's live selfie against the stored profile photo using face-recognition embeddings.

• Provide post-exam reports including score, violation summary, and behaviour timeline to admin and proctors.

• Ensure the entire platform is deployable with a single docker compose up command.

1.5 Scope of the System

The system supports remote proctored exams for university courses, multiple concurrent exam sessions, three user roles (Admin, Proctor, Student), real-time video and audio monitoring with AI analysis, WebSocket-based live alerting, comprehensive event logging, and post-exam result reporting. The platform is deployed as eight Docker containers managed by Docker Compose.

1.6 Platform Overview — Screenshots

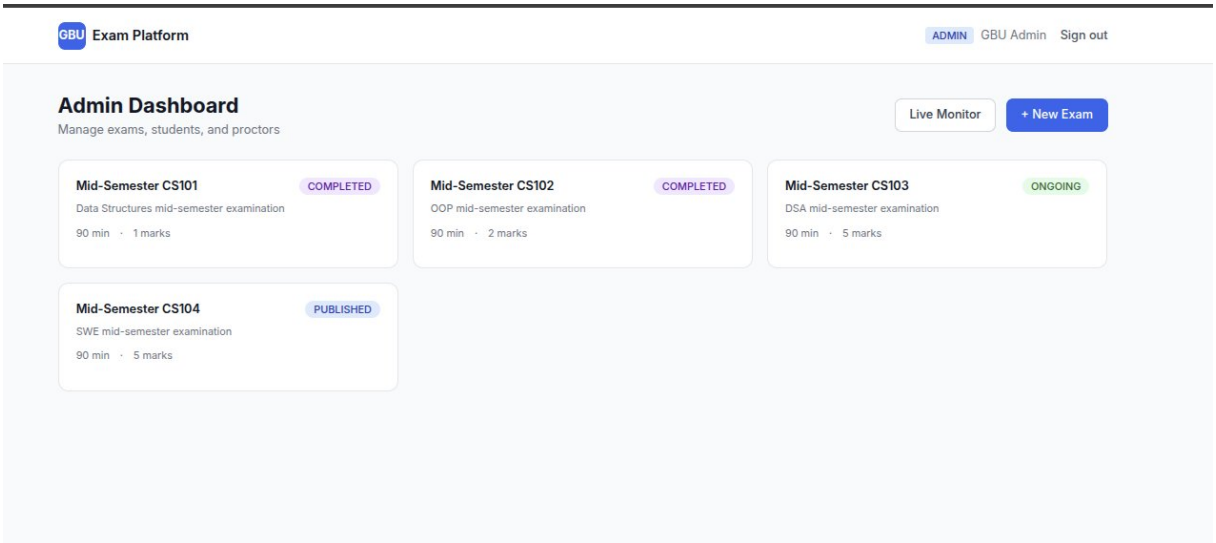


Figure 1 – Admin Dashboard showing exam cards with lifecycle status badges (COMPLETED, ONGOING, PUBLISHED) and controls for Live Monitor and New Exam creation.

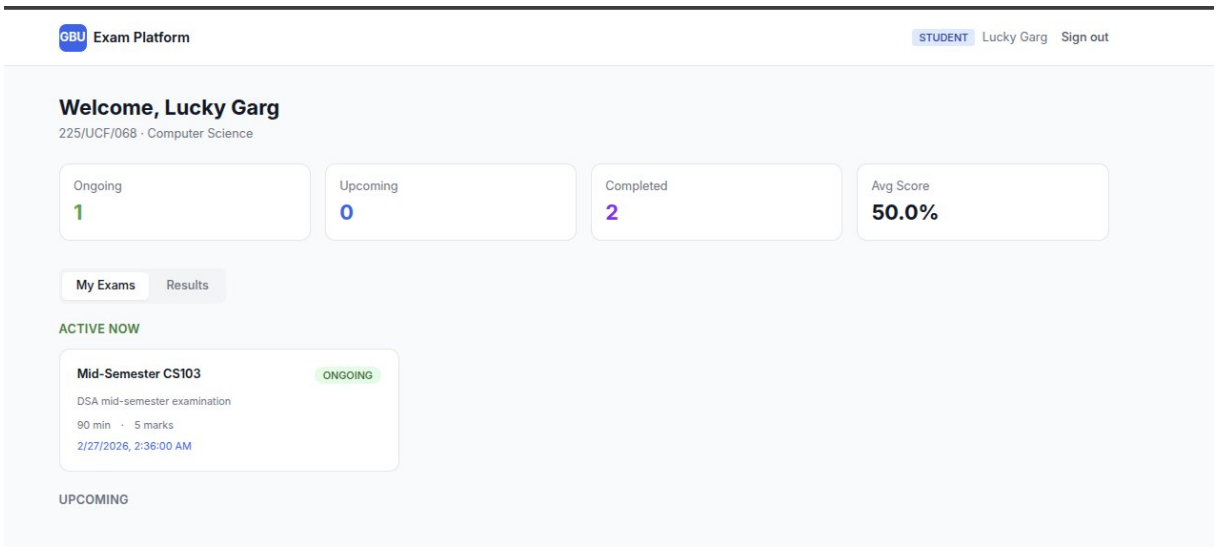


Figure 2 – Student Dashboard (Lucky Garg, 225/UCF/068, Computer Science) showing ongoing, upcoming, and completed exam counters alongside a cumulative average score.

Chapter 2: Literature Review

2.1 Existing Proctoring Systems

Online exam proctoring approaches range from fully manual to semi-automated. Manual video recording requires proctors to review footage after the exam — a 200-student, 2-hour exam would require 400 hours of review time. Commercial platforms such as ProctorU, ExamSoft, and Respondus LockDown Browser offer varying levels of automation but suffer from high per-student licensing costs, proprietary lock-in, and limited customisability. Several universities have faced student backlash over invasive monitoring practices. Basic browser lockdown tools restrict access to other applications but do not monitor student behaviour and can be circumvented through external devices.

2.2 Research in AI-Based Proctoring

Recent research demonstrates the effectiveness of multi-cue AI models. Hossen et al. (2025) created a dataset of 5,500 labelled records of webcam-derived features — face presence, hand position, gaze direction, head pose, and phone presence — and found that ensemble models such as Random Forest and XGBoost achieve high precision and recall for cheating detection. Heinrich's 2025 systematic review advocates for open standards in proctoring technology to ensure interoperability and reduce vendor lock-in. Open-source projects such as Proct-Xam on GitHub have demonstrated the feasibility of building proctoring systems with freely available computer-vision tools.

2.3 Computer Vision in Proctoring

Computer vision plays a central role in modern proctoring systems. MediaPipe FaceMesh provides 468 three-dimensional facial landmarks enabling precise gaze and head-pose tracking. YOLOv8, trained on the COCO dataset, offers real-time detection of prohibited objects including mobile phones and books. The face_recognition library (wrapping Dlib) enables 128-dimensional face embedding comparison for identity verification. These models form the core of the AI proctoring pipeline implemented in this project.

2.4 Gap Analysis

The literature reveals that while individual components of AI-based proctoring have been demonstrated in isolation, no comprehensive open-source platform integrates all detection modules — vision, audio, and behavioural — into a production-ready, fully containerised microservices architecture with real-time alerting, persistent logging, and a complete exam management system. This project fills that gap with a fully implemented and tested platform.

Chapter 3: System Analysis

3.1 Functional Requirements

Student Functionalities

- Register, log in with credentials, and reset password via email OTP.
- View ongoing, upcoming, and completed exams on the student dashboard.
- Review exam rules (camera active, fullscreen required, copy-paste disabled) on the pre-exam page.
- Attempt MCQ questions in a locked browser environment with a live countdown timer.
- Submit the exam manually or have it auto-submitted on timer expiry.
- View score, marks, and submission timestamp on the results page.

Admin / Proctor Functionalities

- Create, edit, publish, and delete exams with configurable title, subject, duration, marks, and schedule.
- Add MCQ questions individually (with question text, four options, correct answer, marks) or via CSV bulk upload.
- Enroll students individually by university roll number or via bulk roll-number paste.
- Monitor all active exam sessions in real time via the live proctor dashboard.
- Receive WebSocket-pushed violation alerts with severity badge, event type, confidence, and timestamp.
- Access per-exam results and export student score data post-exam.

3.2 Non-Functional Requirements

Category	Requirement
Performance	AI detection latency < 2 s; API response < 250 ms; 100+ concurrent sessions
Security	JWT HS512 auth; RBAC on all endpoints; BCrypt password hashing; HTTPS
Scalability	Containerised microservices; stateless APIs; RabbitMQ-based load distribution
Usability	Intuitive dark-theme exam UI; colour-coded alert indicators; minimal student setup
Reliability	Auto-submit on timer expiry; answer auto-save every answer change
Deployability	Single docker compose up command; Flyway-managed DB migrations

3.3 Feasibility Study

Technical Feasibility: The technology stack — Spring Boot, React, Python FastAPI, PostgreSQL, Docker — comprises mature, well-documented, and widely-deployed open-source technologies. Pre-trained ML models (YOLOv8, MediaPipe, Dlib) are freely available and well-tested for real-time applications.

Operational Feasibility: Educational institutions already possess the required infrastructure — internet connectivity and student devices with webcams. Proctors can be trained on the dashboard interface with minimal effort.

Economic Feasibility: The entire system uses open-source components, eliminating licensing costs. Deployment can be performed on institutional servers or low-cost cloud virtual machines, resulting in a total cost of ownership significantly lower than commercial alternatives.

3.4 SWOT Analysis

Strengths	Weaknesses
• Multi-channel AI detection (video, audio, browser behaviour) • Fully open-source; zero licensing costs • Single-command Docker deployment • Privacy-first: only snapshots stored, not raw video	• GPU recommended for optimal YOLOv8 performance • Dependent on student device webcam quality • Continuous internet connectivity required throughout exam
Opportunities	**Threats**
• LMS integration (Moodle, Canvas) • In-person exam monitoring via fixed cameras • Predictive cheating-risk analytics	• Sophisticated evasion techniques • Student privacy concerns • Rapidly evolving AI-assisted cheating tools

Chapter 4: System Design & Architecture

4.1 System Architecture Overview

The platform is composed of **eight Docker containers** communicating over an internal Docker network. This microservices architecture ensures separation of concerns, independent scaling, and technology flexibility. The containers and their roles are as follows:

Container	Port	Description
Nginx	80	Public entry point. Reverse-proxies /api/** and /ws/** to Spring Boot; all other paths to the React SPA.
Frontend	3000 (int.)	Vite-built React 18 SPA served by a Node static server.
Backend	8080	Spring Boot 3.3.4 REST API + WebSocket broker. Handles auth, exam management, enrollment, sessions, and results.
AI Service	8001	Python FastAPI app. Three RabbitMQ consumer threads for frame, audio, and behaviour analysis.
PostgreSQL	5432	Primary relational database. Schema managed by Flyway migrations (V1–V9).
Redis	6379	JWT token blacklist and Spring Session store.
RabbitMQ	5672 / 15672	Async message bus between Spring Boot and the AI service.
MinIO	9000 / 9001	S3-compatible object store for profile photos, question images, and proctoring snapshots.

4.2 Proctoring Data Flow Pipeline

During an active exam session, the student browser continuously captures webcam frames (every ~2 seconds) and audio clips (every ~5 seconds) using HTML5 APIs, and sends these as base64-encoded payloads to the Spring Boot backend via REST. Browser behaviour events (tab switches, fullscreen exits, copy-paste attempts) are pushed via WebSocket. The backend publishes each payload to the corresponding RabbitMQ queue. The AI FastAPI service consumes these queues through three background threads:

- **Frame Consumer** (queue: frame.analysis): Runs MediaPipe BlazeFace for face count, MediaPipe FaceMesh + OpenCV solvePnP for head pose/gaze, and YOLOv8n for object detection.
- **Audio Consumer** (queue: audio.analysis): Runs webrtcvad to detect voice activity in 5–10 second audio clips.
- **Behaviour Consumer** (queue: behavior.events): Applies XGBoost classifier over a sliding window of browser event counts to produce a risk score.

Each consumer publishes its results to the proctoring.results queue, which Spring Boot consumes to persist a ProctoringEvent record in PostgreSQL and push a real-time alert to the proctor dashboard via WebSocket (STOMP over SockJS).

4.3 Backend Architecture (Spring Boot)

The Spring Boot backend follows a layered architecture: **Controller Layer** (REST endpoints, request/response DTOs), **Service Layer** (business logic, validation, workflow rules), **Repository Layer** (Spring Data JPA repositories over PostgreSQL), **Entity Layer** (JPA-mapped domain objects), and **Security Layer** (Spring Security + JWT HS512, BCrypt hashing, RBAC). Database schema evolution is managed automatically by Flyway on application startup. API documentation is auto-generated via SpringDoc OpenAPI and accessible at /swagger-ui.html.

4.4 Frontend Architecture (React)

The frontend is a Vite-bundled React 18 single-page application styled with Tailwind CSS. Axios handles REST API communication with JWT bearer tokens. Browser lockdown is enforced through JavaScript event handlers: visibilitychange (tab switch), fullscreenchange (fullscreen exit), oncopy/onpaste (clipboard), and contextmenu (right-click). WebSocket communication uses STOMP over SockJS for receiving real-time proctor alerts. The exam UI features a dark theme, a question navigator sidebar, a live countdown timer, a camera preview panel, and an auto-save mechanism.

4.5 Database Schema

The relational schema is managed by nine Flyway migrations (V1–V9) and comprises the following core tables:

Table	Key Fields
users	id (UUID), name, email, password_hash, role (ADMIN/PROCTOR/STUDENT), university_roll, department, is_active
exams	id, title, subject, description, start_time, end_time, duration_minutes, total_marks, passing_marks, status, created_by (FK)
questions	id, exam_id (FK), question_text, option_a/b/c/d, correct_option, marks, image_path
exam_enrollments	id, exam_id, user_id, status (REGISTERED / COMPLETED / DISQUALIFIED), enrolled_at
exam_sessions	id, enrollment_id, start_time, end_time, status (IN_PROGRESS / SUBMITTED / TERMINATED), score, risk_score
student_answers	id, session_id, question_id, selected_option, is_correct, answered_at
proctoring_events	id, session_id, event_type, severity, confidence, snapshot_path, risk_score, metadata (JSONB), occurred_at
violation_summaries	id, session_id, total_violations, high_count, critical_count, final_risk_score
behavior_events	id, session_id, event_type (TAB_SWITCH / COPY_PASTE / FULLSCREEN_EXIT / FOCUS_LOSS), occurred_at, metadata (JSONB)

4.6 Security Architecture

Authentication uses JWT HS512 tokens with a 1-hour access token and 7-day refresh token. Logout blacklists the current token in Redis. All passwords are hashed with BCrypt. Role-based access control is enforced on every API endpoint via Spring Security. All REST communication

is transmitted over HTTPS; internal Docker service communication is isolated within the container network. SQL injection is prevented through parameterised JPA queries.

Chapter 5: Platform Modules & Features

5.1 Authentication & Authorisation

The platform supports three roles: ADMIN, PROCTOR, and STUDENT. Users log in with email and password; the backend returns a JWT access token (1 hour) and refresh token (7 days). Password reset is available via email OTP. Refresh token rotation is enforced, and logout blacklists the current token in Redis, preventing reuse.

5.2 Exam Management

Administrators can create exams by specifying title, subject, description, start and end time, duration (minutes), total marks, and passing marks. Each exam progresses through a defined lifecycle: **DRAFT** → **PUBLISHED** → **ONGOING** → **COMPLETED**. In the DRAFT state, admins can edit all metadata; PUBLISHED makes the exam visible to enrolled students; ONGOING activates the proctoring pipeline; COMPLETED locks the exam and finalises results.

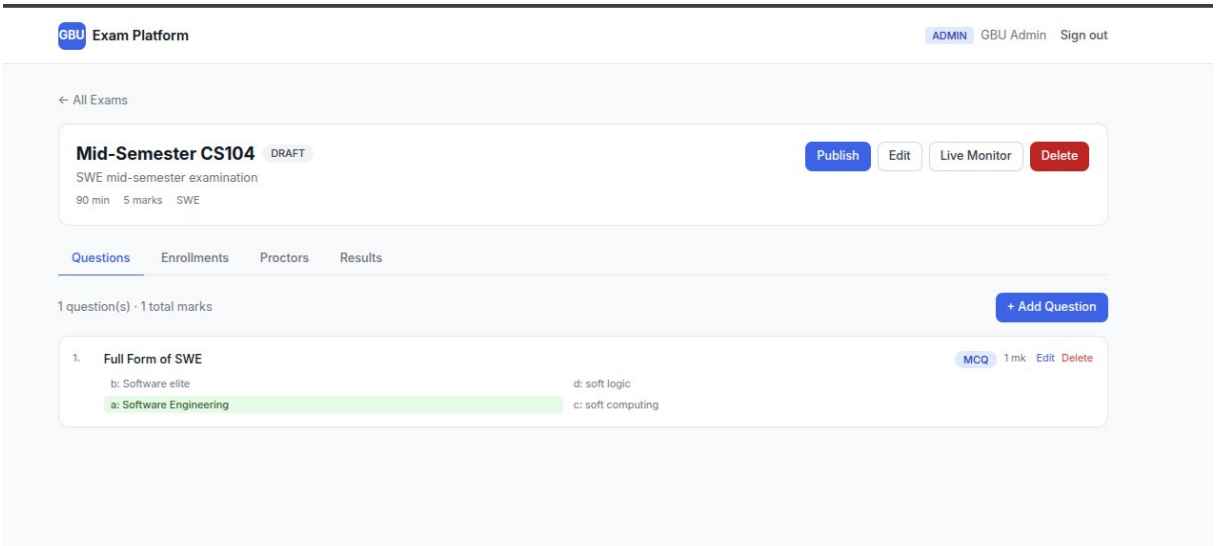


Figure 3 – Exam detail page for Mid-Semester CS104 (SWE) in DRAFT state, showing Questions, Enrollments, Proctors, and Results tabs with the Add Question and Publish controls.

5.3 Question Management

MCQ questions are added via a modal dialogue specifying: question text, marks per question, four labelled options (a–d), and the correct answer. Questions can also be bulk-uploaded via CSV. Question images are stored in MinIO. Each question is independently editable and deletable before the exam is published.

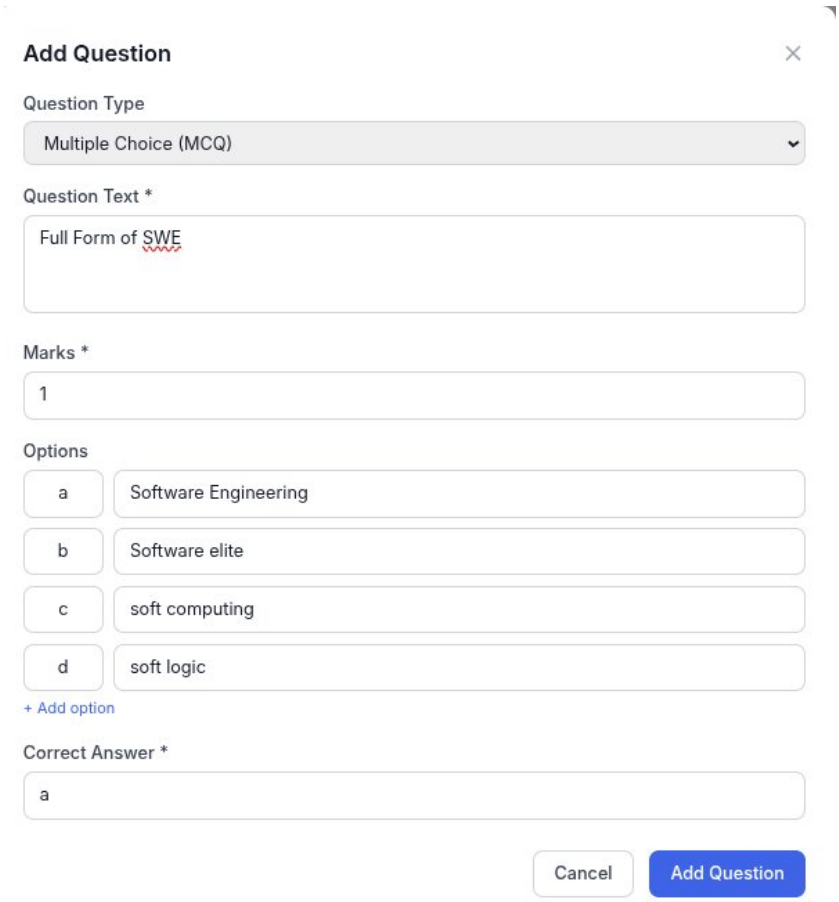


Figure 4 – Add Question modal showing MCQ configuration with four options and correct answer designation. The highlighted option (a: Software Engineering) indicates the correct answer.

5.4 Student Exam Flow

Students access their personalised dashboard displaying all assigned exams categorised as Active Now, Upcoming, or Completed, along with a cumulative average score. Selecting an ongoing exam opens the pre-exam information page, which displays exam metadata (duration, total marks, passing marks, subject, start/end time) and a prominent proctoring rules notice. The student must explicitly acknowledge these rules before the exam session begins.

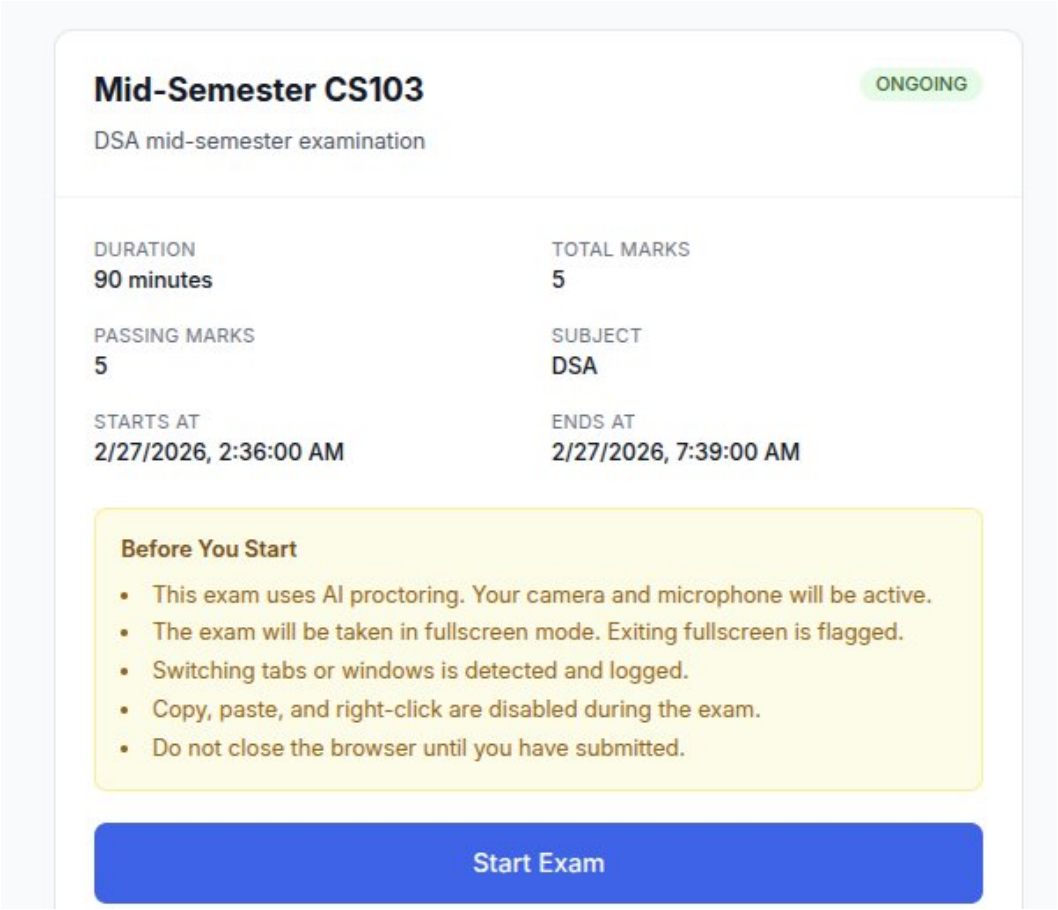


Figure 5 – Pre-exam information page for Mid-Semester CS103 (DSA). The "Before You Start" panel enumerates proctoring conditions: active camera and microphone, fullscreen enforcement, tab-switch detection, disabled copy-paste, and browser-close warning.

Once the student clicks Start Exam, the browser requests webcam and microphone permissions and enters fullscreen mode. The exam interface features a dark theme for reduced eye strain, a question navigator sidebar (colour-coded: green for answered, grey for not answered), the question text and options in the centre panel, a live countdown timer and answer counter in the top bar, and a live camera preview labelled "Proctored" in the top-right corner. Answers are auto-saved on each selection. The student can navigate between questions freely and submit manually or allow the timer to auto-submit.

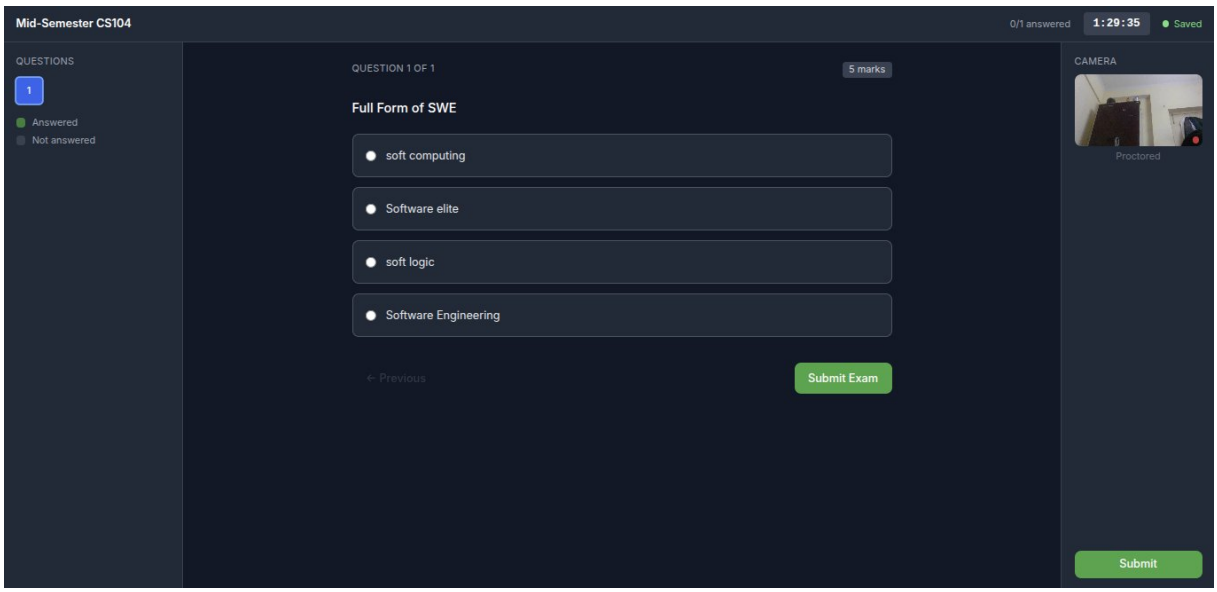
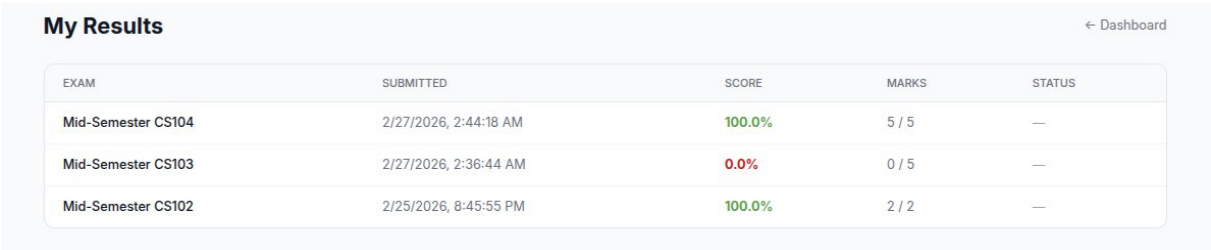


Figure 6 – Active exam interface (Mid-Semester CS104, SWE). The left sidebar shows question navigation status; the top bar displays the progress counter (0/1 answered) and countdown timer (1:29:35); the camera feed with "Proctored" label is visible in the top-right corner.

5.5 Results & Reporting

Upon submission, marks are calculated server-side: for each question, the student's selected option is compared against the stored correct answer. Score is computed as (marks obtained / total marks) × 100. Results are immediately available on the student's results page, displaying exam name, submission timestamp, score percentage (green for pass, red for fail), raw marks obtained, and pass/fail status.



My Results ← Dashboard

EXAM	SUBMITTED	SCORE	MARKS	STATUS
Mid-Semester CS104	2/27/2026, 2:44:18 AM	100.0%	5 / 5	—
Mid-Semester CS103	2/27/2026, 2:36:44 AM	0.0%	0 / 5	—
Mid-Semester CS102	2/25/2026, 8:45:55 PM	100.0%	2 / 2	—

Figure 7 – My Results page for student Lucky Garg (225/UCF/068). Mid-Semester CS104 shows 100.0% (5/5), CS103 shows 0.0% (0/5), and CS102 shows 100.0% (2/2), with submission timestamps.

Chapter 6: AI Proctoring & Detection Modules

The AI proctoring service is a Python FastAPI application running three independent RabbitMQ consumer threads. Each thread consumes messages from a dedicated queue, performs inference using the appropriate ML model, and publishes a proctoring result back to Spring Boot via the proctoring.results queue. The following subsections describe each detection module in detail.

6.1 Identity Verification

Before the exam session begins, the student submits a live selfie via the webcam. The AI service uses the **face_recognition** library (wrapping Dlib) to extract a 128-dimensional face embedding from the live image and compares it against the stored profile photo embedding retrieved from MinIO. If the Euclidean distance between the two embeddings exceeds 0.6, access is denied, preventing impersonation.

6.2 Face Presence Detection

The Frame Consumer processes each webcam frame using **MediaPipe BlazeFace** to count the number of faces detected. Exactly one face is expected throughout the exam. Detection of zero faces (FACE_NOT_DETECTED) suggests the student has left the exam area; detection of two or more faces (MULTIPLE_FACES) indicates possible assistance from another person. Both events generate a HIGH-severity proctoring event.

6.3 Gaze Tracking and Head Pose Estimation

MediaPipe FaceMesh (468 3D landmarks) is used to extract key facial landmarks. OpenCV's solvePnP with a standard 3D face model computes head orientation (pitch, yaw, roll). Large yaw values (looking left or right) and large pitch values (looking down) sustained beyond configurable thresholds generate a GAZE_AWAY event. This detects students reading from notes, communicating with another person, or consulting a second screen.

6.4 Lip Movement Detection

Using the FaceMesh lip landmark coordinates, the AI service computes a lip-gap ratio — the distance between the upper and lower lip landmarks normalised by face height. Sustained mouth opening beyond the threshold indicates the student may be talking, dictating answers, or communicating with an accomplice. This module contributes 10% to the composite risk score.

6.5 Object Detection (YOLOv8n)

The Frame Consumer runs **YOLOv8n** (Ultralytics) — pretrained on the COCO dataset — on each webcam frame to detect prohibited objects. Relevant COCO classes include "cell phone" (PHONE_DETECTED), "book" (NOTES_DETECTED), and "person" (MULTIPLE_PERSONS when count > 1). Detections above a confidence threshold of 0.5 generate a CRITICAL-severity proctoring event and a snapshot is saved to MinIO for proctor review.

6.6 Voice Activity Detection

The Audio Consumer receives 5–10 second base64-encoded audio clips and processes them through **webrtcvad** (Google's WebRTC Voice Activity Detector). The audio is decoded, resampled to 16 kHz mono PCM if necessary, and framed into 30 ms windows. If more than a

configurable proportion of frames contain speech, a SUSPICIOUS_AUDIO event is generated. In examinations where students are expected to remain silent, any detected speech triggers a MEDIUM-severity alert.

6.7 Browser Behaviour Monitoring

The React frontend captures browser behaviour events using JavaScript event listeners and sends them to the Spring Boot backend via WebSocket, which then publishes them to the behavior.events RabbitMQ queue. The Behaviour Consumer collects events over a sliding window and feeds event counts as features to an **XGBoost** binary classifier. Monitored events include TAB_SWITCH (visibilitychange), FULLSCREEN_EXIT (fullscreenchange), COPY_PASTE (oncopy/onpaste), CONTEXT_MENU (contextmenu), and FOCUS_LOSS. A risk score above 0.4 generates a SUSPICIOUS_BEHAVIOR event.

6.8 Composite Risk Score Computation

Each detection module outputs a normalised score between 0 and 1. These scores are combined using a configurable weighted formula to produce a composite risk score:

```
Risk = 0.30 × Face + 0.20 × Gaze + 0.20 × Objects + 0.10 × Mouth +
0.20 × Behaviour
```

Risk Score Range	Severity Level	Action
< 0.40	LOW	Suppressed (no alert generated)
0.40 – 0.74	MEDIUM	Proctoring event logged; proctor notified
0.75 – 0.89	HIGH	High-priority alert with snapshot
≥ 0.90	CRITICAL	Critical alert; snapshot saved; session flagged

6.9 Violation Types Summary

Event Type	Source Module	Severity
FACE_NOT_DETECTED	Face Presence (MediaPipe)	HIGH
MULTIPLE_FACES	Face Presence (MediaPipe)	HIGH
GAZE_AWAY	Head Pose (FaceMesh + solvePnP)	MEDIUM
PHONE_DETECTED	Object Detection (YOLOv8n)	CRITICAL
NOTES_DETECTED	Object Detection (YOLOv8n)	CRITICAL
MULTIPLE_PERSONS	Object Detection (YOLOv8n)	CRITICAL
SUSPICIOUS_AUDIO	Voice Activity (webrtcvad)	MEDIUM
SUSPICIOUS_BEHAVIOR	Browser Events (XGBoost)	MEDIUM–HIGH

Chapter 7: Technology Stack

Layer	Technology	Version / Details
Backend API	Spring Boot	3.3.4 (Java 17)
Frontend	React + Vite + Tailwind CSS	React 18
AI / Proctoring Service	Python FastAPI + Uvicorn	Python 3.11
Database	PostgreSQL	16
Cache / Session Store	Redis	7
Message Broker	RabbitMQ	3.13 with Management Plugin
Object Storage	MinIO	Latest stable
Reverse Proxy	Nginx	1.25
Containerisation	Docker + Docker Compose	Compose v3.9
DB Migrations	Flyway	Managed by Spring Boot auto-startup
Real-time Push	WebSocket (STOMP over SockJS)	—
Authentication	JWT HS512 via Spring Security	jjwt 0.11.5
API Documentation	SpringDoc OpenAPI / Swagger UI	2.3.0
Face Detection	MediaPipe BlazeFace	mediapipe 0.10
Gaze / Head Pose	MediaPipe FaceMesh + OpenCV solvePnP	opencv-python 4.x
Object Detection	YOLOv8n	ultralytics 8.x
Identity Verification	face_recognition (Dlib)	face-recognition 1.3
Voice Activity	webrtcvad	2.0.10
Behaviour Scoring	XGBoost classifier	xgboost 2.x

7.1 API Endpoints Reference

The backend exposes RESTful endpoints documented via Swagger UI at /swagger-ui.html. Key endpoint groups are summarised below:

Method	Endpoint	Description
POST	`/api/auth/login`	Authenticate user; return JWT access + refresh tokens
POST	`/api/auth/refresh`	Exchange refresh token for new access token
POST	`/api/auth/logout`	Blacklist current token in Redis
GET	`/api/exams`	List exams (paginated, role-filtered)
POST	`/api/exams`	Create exam (Admin)
POST	`/api/exams/{id}/publish`	Publish exam (Admin)
POST	`/api/exams/{id}/enrollments`	Enroll student by roll number

POST	`/api/exams/{id}/enrollments/bulk`	Bulk enroll by roll numbers
POST	`/api/sessions/start/{examId}`	Student starts exam session
POST	`/api/sessions/{id}/submit`	Submit exam and calculate results
POST	`/api/proctoring/frame`	Upload webcam frame for AI analysis
POST	`/api/proctoring/audio`	Upload audio clip for AI analysis
POST	`/api/proctoring/behavior`	Log browser behaviour event
GET	`/api/proctoring/sessions/{id}/events`	Get proctoring events for a session
GET	`/api/reports/students/{userId}/history`	Student's own exam history
GET	`/api/reports/exams/{examId}/results`	All results for an exam (Admin/Proctor)

Chapter 8: Security, Privacy & Ethics

8.1 Data Privacy and Minimisation

The platform does not store continuous raw video recordings. Instead, only discrete JPEG snapshots associated with specific violation events are retained in MinIO under the proctoring-snapshots/ bucket. This approach significantly reduces both storage requirements and privacy exposure, retaining only the evidence directly relevant to flagged events. Retention policies govern the automatic deletion of snapshots after a configurable review period.

8.2 Consent and Transparency

Students are presented with a clear, prominent proctoring rules notice on the pre-exam page before every session. The notice explicitly states that the camera and microphone will be active, that fullscreen mode is required, and that tab switches and copy-paste actions are monitored. Students must affirmatively click "Start Exam" to consent to these conditions. No monitoring begins before this explicit acknowledgement.

8.3 Secure Data Transmission

All REST API communication is transmitted over HTTPS. WebSocket connections for real-time events use STOMP over SockJS with token-based authentication. Internal service-to-service communication (Spring Boot to RabbitMQ, PostgreSQL, Redis) occurs within the isolated Docker internal network, not exposed to external traffic. Input validation and parameterised JPA queries prevent SQL injection.

8.4 Ethical Considerations

All proctoring flags generated by the AI modules are advisory. No academic penalty is applied automatically; every violation requires review by a human proctor or administrator. Each event record includes the event type, confidence score, timestamp, and an optional snapshot, providing the evidentiary basis for a fair, transparent, and contestable decision-making process. Detection thresholds are calibrated to suppress LOW-severity events, reducing the frequency of false positives from momentary innocuous actions.

Chapter 9: Evaluation & Testing

9.1 Testing Approach

The platform was evaluated through a combination of unit testing, manual API testing, and end-to-end functional testing.

- **Unit Tests:** JUnit 5 + Spring Boot Test for backend service and repository layers. Pytest for AI service modules including test_audio_vad.py, test_risk_aggregator.py, and test_object_detector.py.
- **API Testing:** All REST endpoints verified via Swagger UI (/swagger-ui.html). Request/response schemas validated against the OpenAPI specification.
- **End-to-End Testing:** Complete exam lifecycle tested in browser — admin creates exam, enrolls student, publishes; student joins, answers, submits; results verified server-side.
- **Proctoring Pipeline Testing:** Simulated violation scenarios (phone in frame, tab switch, exiting fullscreen, speaking) verified to produce correct proctoring events in PostgreSQL and real-time alerts on the proctor dashboard.

9.2 Evaluation Metrics

Metric	Target	Notes
AI detection latency	< 2 seconds	Frame-to-alert round trip via RabbitMQ
API response time	< 250 ms	Measured on REST endpoints under normal load
False alarm rate	LOW severity suppressed	Thresholds tuned to ignore momentary events
Audit traceability	100%	Every event traceable to session, student, and timestamp
Exam submission accuracy	100%	Server-side marking verified against correct_option

Chapter 10: Future Scope

LMS Integration: Future versions can integrate with popular Learning Management Systems such as Moodle, Canvas, and Blackboard, enabling seamless exam scheduling, grade synchronisation, and student enrollment from existing academic systems.

Advanced Speech Recognition: The audio pipeline can be extended with the Vosk (Kaldi-based) model to transcribe detected speech in real time, enabling detection of prohibited keyword utterances.

Predictive Risk Analytics: Historical proctoring event data can be used to train predictive models that identify students at higher risk of cheating before the exam begins, enabling proactive intervention.

Multi-language Support: The frontend can be internationalised to support Hindi and other regional languages, broadening accessibility for students across India.

Mobile Proctor Application: A dedicated mobile application for proctors would enable monitoring from any location, with push notifications for CRITICAL alerts and the ability to terminate sessions remotely.

Multi-Institution Deployment: The platform can be extended to support multi-tenant deployment, where multiple institutions share infrastructure while maintaining strict data isolation, reducing per-institution deployment costs.

Adaptive Proctoring Intensity: The system could dynamically adjust monitoring intensity based on the ongoing risk score — increasing frame-analysis frequency for high-risk sessions and relaxing it for low-risk ones.

Chapter 11: Conclusion

This report presents the complete design, architecture, and implementation of an **AI-Driven Proctored Exam Platform** built for Gautam Buddha University. The system addresses the critical challenge of maintaining examination integrity in online settings by delivering a fully functional, production-grade platform that combines a robust exam management system with a real-time AI proctoring pipeline.

The platform was implemented as eight Docker containers orchestrated by Docker Compose, with a Spring Boot 3.3.4 backend, a React 18 frontend, and a Python FastAPI AI service. The proctoring pipeline integrates YOLOv8n for object detection, MediaPipe FaceMesh with OpenCV solvePnP for gaze and head-pose tracking, webrtcvad for voice activity detection, the face_recognition library for identity verification, and XGBoost for behavioural risk scoring. All seven AI-detectable violation types are implemented, logged to PostgreSQL, and pushed to the proctor dashboard via WebSocket in real time.

The use of entirely open-source technologies ensures the solution is accessible, cost-effective, and free from vendor dependencies. The modular architecture allows each component to be independently developed, tested, and scaled. The platform has been functionally validated through end-to-end exam sessions, demonstrating correct operation of the exam lifecycle, proctoring event generation, real-time alerting, and server-side result calculation.

The AI-Driven Proctored Exam Platform stands as a scalable, reliable, and secure system well-suited to the requirements of modern educational institutions seeking an open-source alternative to expensive commercial proctoring solutions.

References

1. Hossen, M. I. et al. (2025). Students Suspicious Behaviors Detection Dataset for AI-Powered Online Exam Proctoring. Mendeley Data. https://data.mendeley.com/datasets/39xs8th543/1

2. Heinrich, S. (2025). A Systematic-Narrative Review of Online Proctoring Systems and a Case for Open Standards. Open Praxis, 17(3). https://openpraxis.org/articles/10.55982/openpraxis.17.3.836

3. Ultralytics. (2023). YOLOv8: A New Frontier in Object Detection. https://docs.ultralytics.com

4. Lugaresi, C. et al. (2019). MediaPipe: A Framework for Building Perception Pipelines. arXiv:1906.08172. https://mediapipe.dev

5. King, D. E. (2009). Dlib-ml: A Machine Learning Toolkit. Journal of Machine Learning Research, 10, 1755–1758. http://dlib.net

6. Grinberg, M. (2018). Flask Web Development (2nd ed.). O'Reilly Media. — FastAPI: https://fastapi.tiangolo.com

7. Spring Framework Team. (2024). Spring Boot Reference Documentation v3.3. https://docs.spring.io/spring-boot

8. React Team. (2023). React 18 Documentation. https://react.dev

9. RabbitMQ Team. (2024). RabbitMQ Documentation. https://www.rabbitmq.com/docs

10. MinIO Team. (2024). MinIO Object Storage Documentation. https://min.io/docs

11. Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools. https://opencv.org

12. Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD 2016. arXiv:1603.02754

13. Wiseman, J. (2017). webrtcvad: Python interface to the WebRTC VAD library. https://github.com/wiseman/py-webrtcvad

14. Roboflow Universe. (2024). Cheating Detection Datasets and Models. https://universe.roboflow.com/search?q=class:cheating